
Bài giảng

Các kiến trúc Deep Learning sau CNN

Từ ResNet đến Vision Transformer

Phần tiếp theo của bài giảng CNN cổ điển

Giảng viên: Hà Minh Tuấn

Ngày 21 tháng 5 năm 2026

Tóm tắt nội dung

CNN cổ điển (LeNet, AlexNet, VGG) đã thay đổi hoàn toàn ngành **Computer Vision**, nhưng vẫn còn ba hạn chế lớn: (1) *khó train mạng rất sâu* do vanishing gradient, (2) *receptive field cục bộ*, không nhìn được context xa, và (3) *chỉ làm được phân loại*, chưa giải quyết được các bài toán phức tạp như detection, segmentation.

Bài giảng này trình bày **8 kiến trúc then chốt** đã giải quyết từng hạn chế đó, theo trình tự thời gian từ 2015 đến 2021: ResNet, DenseNet, U-Net, Faster R-CNN, YOLO, Mask R-CNN, Vision Transformer, và Swin Transformer. Mỗi mạng được trình bày theo bốn câu hỏi cốt lõi: *Vấn đề trước đó là gì?*, *Ý tưởng cốt lõi?*, *Giải bài toán nào?*, và *Vì sao thắng?*

Triết lý xuyên suốt: “Mỗi kiến trúc mới ra đời để giải quyết một hạn chế cụ thể của kiến trúc trước đó. Hiểu vấn đề trước khi hiểu giải pháp.”

Yêu cầu tiên quyết: Sinh viên đã nắm được kiến thức CNN cổ điển (convolution, pooling, BatchNorm, skip connection cơ bản), MLP, và Transformer (self-attention, multi-head attention).

Mục lục

1	Hai mạng tiền đề: AlexNet và VGGNet	4
1.1	AlexNet (2012) — Khoảnh khắc khai sinh Deep Learning hiện đại	4
1.1.1	Bối cảnh ra đời	4
1.1.2	Nỗi đau mà AlexNet giải quyết	4
1.1.3	Kiến trúc AlexNet	4
1.1.4	Bốn đóng góp đột phá của AlexNet	5
1.1.5	Hạn chế của AlexNet	5
1.2	VGGNet (2014) — Triết lý “đơn giản nhưng sâu”	6

1.2.1	Bối cảnh ra đời	6
1.2.2	Nỗi đau mà VGGNet giải quyết	6
1.2.3	Triết lý “3×3”	7
1.2.4	Kiến trúc VGG-16	7
1.2.5	Đóng góp của VGGNet	7
1.2.6	Hạn chế của VGGNet	8
1.3	So sánh AlexNet và VGGNet — Bảng tổng hợp	9
1.4	Hai nỗi đau lớn còn lại sau VGGNet	9
2	Bức tranh toàn cảnh: Tại sao cần kiến trúc mới?	9
2.1	Ba hạn chế lớn của CNN cổ điển	10
2.2	Dòng chảy lịch sử của 8 kiến trúc	10
3	Nhóm 1 — Cải tiến chính CNN (2014–2017)	10
3.1	ResNet (2015) — Cuộc cách mạng “skip connection”	10
3.1.1	Vấn đề trước đó	10
3.1.2	Ý tưởng cốt lõi — Residual Learning	11
3.1.3	Minh họa Residual Block	11
3.1.4	Giải bài toán nào?	12
3.1.5	Vì sao thắng?	12
3.2	DenseNet (2017) — Cực đoan hóa skip connection	12
3.2.1	Vấn đề và Ý tưởng	12
3.2.2	Minh họa Dense Block	13
3.2.3	Giải bài toán nào?	13
3.2.4	So sánh ResNet và DenseNet	14
4	Nhóm 2 — Vượt qua phân loại: Detection & Segmentation	14
4.1	Ba câu hỏi của Computer Vision	14
4.2	U-Net (2015) — Vua của Segmentation	14
4.2.1	Vấn đề trước đó	14
4.2.2	Ý tưởng cốt lõi — Encoder-Decoder + Skip	15
4.2.3	Minh họa kiến trúc U-Net	15
4.2.4	Giải bài toán nào?	16
4.2.5	Vì sao thắng?	16
4.3	Mô hình R-CNN (Regions with CNN features)	16
4.4	Mô hình Fast R-CNN	18
4.5	Faster R-CNN (2015) — Detection chính xác cao	19
4.5.1	Vấn đề trước đó	19
4.5.2	Ý tưởng cốt lõi — Two-Stage Detection	19
4.5.3	Giải bài toán nào?	20
4.5.4	Vì sao thắng (và vì sao có giới hạn)?	20
4.6	YOLO (2016, hiện đã đến v8/v11) — Detection Real-time	20
4.6.1	Vấn đề và Ý tưởng	21
4.6.2	Giải bài toán nào?	21
4.6.3	Trade-off giữa Faster R-CNN và YOLO	21
4.7	Mask R-CNN (2017) — Instance Segmentation	22
4.7.1	Vấn đề và Ý tưởng	22
4.7.2	Giải bài toán nào?	22
4.7.3	Phân biệt 3 loại segmentation	23

5	Nhóm 3 — Cuộc cách mạng Transformer trong Vision (2020+)	23
5.1	Vision Transformer (ViT, 2020) — CNN không còn là duy nhất	23
5.1.1	Vấn đề trước đó	23
5.1.2	Ý tưởng cốt lõi — Patch như Word	23
5.1.3	Minh họa pipeline ViT	24
5.1.4	Giải bài toán nào?	24
5.1.5	Vì sao ViT thắng CNN khi đủ data?	24
5.1.6	Hạn chế của ViT	24
5.2	Swin Transformer (2021) — Transformer “biết tiết kiệm”	24
5.2.1	Vấn đề của ViT	25
5.2.2	Ý tưởng cốt lõi — Windowed Attention + Shift	25
5.2.3	Minh họa Shifted Window	25
5.2.4	Giải bài toán nào?	25
5.2.5	Vì sao Swin tốt hơn ViT cho nhiều bài toán?	26
6	Tổng hợp: Bảng so sánh 8 kiến trúc	26
6.1	Khi nào dùng kiến trúc nào? — Cây quyết định	27
6.2	Bài học triết lý quan trọng nhất	27
7	Code minh họa: Sử dụng các mô hình pretrained	27
8	Bài tập tổng hợp	29
8.1	Bài tập lý thuyết	29
8.2	Bài tập tình huống	29
8.3	Bài tập thực hành	30
9	Tổng kết	30
9.1	Sơ đồ tư duy cuối cùng	30
9.2	Lời khuyên	31

1. Hai mạng tiền đề: AlexNet và VGGNet

Ghi chú

Trước khi đi vào các kiến trúc sau CNN cổ điển (ResNet, ViT, ...), sinh viên cần hiểu rõ **hai mạng đã định hình cả ngành Computer Vision hiện đại**: AlexNet (2012) và VGGNet (2014). Đây là hai kiến trúc tiền đề — chúng đặt ra cả “hình mẫu” lẫn các *vấn đề* mà thế hệ sau (ResNet, DenseNet) phải giải quyết.

1.1 AlexNet (2012) — Khoảnh khắc khai sinh Deep Learning hiện đại

Bối cảnh ra đời

Đầu thập niên 2010, **Computer Vision vẫn bị thống trị bởi các phương pháp thủ công** như SIFT, HOG, kết hợp với SVM. Mỗi năm, cuộc thi **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** thu hút các nhóm nghiên cứu hàng đầu nhưng kết quả cải thiện rất chậm — top-5 error rate quanh quẩn ở mức 26%.

Năm 2012, ba nhà nghiên cứu tại Đại học Toronto — *Alex Krizhevsky, Ilya Sutskever*, và *Geoffrey Hinton* — nộp một mô hình CNN sâu vào ILSVRC. Kết quả là một **cơn địa chấn**: AlexNet đạt top-5 error rate **15.3%**, vượt người về nhì (26.2%) tới hơn 10 điểm phần trăm.

Câu hỏi

Tại sao một kiến trúc CNN tương đối “đơn giản” (5 conv + 3 FC) lại có thể *nhảy vọt* hơn 10% so với các phương pháp thủ công đã được nghiên cứu suốt 20 năm? Cái gì *thực sự* mới?

Nỗi đau mà AlexNet giải quyết

Khái niệm cốt lõi

Trước AlexNet, có ba “nỗi đau” lớn:

1. **Feature engineering thủ công**: Phải thiết kế tay từng feature (SIFT, HOG) — tốn công, phụ thuộc vào chuyên gia, và không tổng quát hóa được.
2. **Không scale được lên dataset lớn**: SVM với kernel không scale lên hàng triệu ảnh.
3. **Mạng nông không học được biểu diễn phong phú**: LeNet (1998) chỉ dùng cho MNIST 28×28 — không xử lý được ảnh tự nhiên 224×224 với 1000 class.

Kiến trúc AlexNet

AlexNet gồm **8 layer học được**: 5 convolutional + 3 fully connected, tổng cộng **60 triệu tham số** và 650.000 neurons. Ảnh đầu vào $224 \times 224 \times 3$ được xử lý qua các tầng sau,

với spatial dimension *giảm dần* và channels *tăng dần*: $224 \rightarrow 55 \rightarrow 27 \rightarrow 13$, channels $3 \rightarrow 96 \rightarrow 256 \rightarrow 384$.

Ví dụ minh họa

Xem hình minh họa kiến trúc AlexNet:

- Sơ đồ gốc từ paper Krizhevsky et al. (2012): https://en.wikipedia.org/wiki/AlexNet#/media/File:Comparison_image_neural_networks.svg
- Sơ đồ học thuật trên ResearchGate: https://www.researchgate.net/figure/Architecture-of-AlexNet-Krizhevsky-et-al-2012_fig2_337486420

Bốn đóng góp đột phá của AlexNet

Insight cần đạt

AlexNet không phải vô tình mà thắng — nó đem đến **bốn cải tiến cốt lõi**, mỗi cái đều ảnh hưởng đến mọi mạng CNN sau này:

1. **ReLU thay vì sigmoid/tanh**: Giải quyết vanishing gradient ở mức cơ bản và tăng tốc training gấp 6 lần.
2. **Training trên GPU** (cụ thể là 2 GPU NVIDIA GTX 580 song song): Lần đầu tiên một mạng deep được train trong vài ngày thay vì vài tháng.
3. **Dropout**: Kỹ thuật regularization mới, giúp mạng sâu không bị overfit dù có 60M tham số.
4. **Data augmentation**: Flip, crop, PCA color jitter — biến dataset 1.2M ảnh thành “vô hạn” samples khác nhau.

Hạn chế của AlexNet

Lưu ý quan trọng

Dù mang tính cách mạng, AlexNet vẫn có nhiều hạn chế:

- **Kiến trúc “chấp vá”**: dùng kernel kích thước không đồng đều (11×11 , 5×5 , 3×3) — khó phân tích lý thuyết.
- **Kernel 11×11 ở layer đầu quá lớn**: nhiều tham số, mất chi tiết spatial.
- **Local Response Normalization (LRN)**: kỹ thuật normalize thiếu hiệu quả, sau này bị BatchNorm thay thế hoàn toàn.
- **Không scale được sâu hơn**: thử stack 10–20 layer kiểu AlexNet thì *accuracy giảm* thay vì tăng.
- **60M tham số**: phần lớn nằm ở 3 FC layer cuối $\Rightarrow$ tốn memory.

1.2 VGGNet (2014) — Triết lý “đơn giản nhưng sâu”**Bối cảnh ra đời**

Sau AlexNet, cộng đồng đặt câu hỏi: “*Liệu chỉ cần tăng số layer là sẽ tốt hơn? Hay phải thiết kế các kernel size đặc biệt như AlexNet?*”

Năm 2014, hai nhà nghiên cứu *Karen Simonyan* và *Andrew Zisserman* từ **Visual Geometry Group** (Oxford) công bố paper “*Very Deep Convolutional Networks for Large-Scale Image Recognition*”. Họ đưa ra một câu trả lời **đơn giản đến ngạc nhiên**: chỉ dùng duy nhất $\text{conv } 3 \times 3$, nhưng *stack thật sâu*.

VGGNet về nhì tại ILSVRC 2014 (sau GoogLeNet) nhưng *quan trọng hơn về mặt sư phạm* — kiến trúc rõ ràng, đẹp về mặt thiết kế, dễ phân tích và *cực kỳ dễ implement*.

Nỗi đau mà VGGNet giải quyết**Khái niệm cốt lõi**

VGGNet trả lời ba câu hỏi mà AlexNet để lại:

1. “**Phải dùng kernel lớn (11×11 , 7×7) như AlexNet không?**”
 $\Rightarrow$ Không. Stack nhiều $\text{conv } 3 \times 3$ tương đương về receptive field nhưng ít tham số hơn và nhiều nonlinearity hơn.
2. “**Tăng độ sâu có giúp gì không?**”
 $\Rightarrow$ Có. Từ VGG-11 đến VGG-19, *accuracy tăng đều* theo độ sâu.
3. “**Có thiết kế nào đơn giản, có hệ thống không?**”
 $\Rightarrow$ Có. VGG dùng *cùng một block đơn vị* ($\text{conv } 3 \times 3 + \text{ReLU}$) lặp đi lặp lại
 $\Rightarrow$ vô cùng dễ hiểu, dễ code.

Triết lý “3×3”

Đây là điểm sáng tạo **quan trọng nhất** của VGGNet — một insight đến nay vẫn được dùng trong ResNet, DenseNet, và thậm chí Swin Transformer.

Ví dụ minh họa

So sánh: một conv 7×7 vs ba conv 3×3 stack liên tiếp:

- Cả hai đều có **receptive field** 7×7 .
- Conv 7×7 với C channels: $7 \cdot 7 \cdot C \cdot C = 49C^2$ tham số.
- Ba conv 3×3 : $3 \cdot (3 \cdot 3 \cdot C \cdot C) = 27C^2$ tham số — giảm $\sim 45\%$.
- Ba conv 3×3 có **3 lần ReLU** thay vì 1 $\Rightarrow$ expressive hơn rất nhiều.

Kiến trúc VGG-16

VGG-16 gồm **13 conv 3×3 + 3 FC**, tổ chức thành 5 “khối” (block):

- Block 1–2: mỗi block có 2 conv + 1 max pool.
- Block 3–5: mỗi block có 3 conv + 1 max pool.
- Cuối: Flatten $\rightarrow$ FC(4096) $\rightarrow$ FC(4096) $\rightarrow$ FC(1000) $\rightarrow$ Softmax.

Spatial dimension giảm theo cấp số nhân: $224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$. Channels tăng theo cấp số nhân: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$. **Đây là pattern thiết kế chuẩn** mà mọi CNN sau này đều theo.

Ví dụ minh họa

Xem hình minh họa kiến trúc VGG-16:

- Sơ đồ học thuật trên ResearchGate: https://www.researchgate.net/figure/VGG-16-architecture-Simonyan-Zisserman-2014_fig2_381230340
- Hướng dẫn tổng quát kèm các biến thể VGG-11/13/16/19: <https://viso.ai/deep-learning/vgg-very-deep-convolutional-networks/>
- Paper gốc (tham khảo Bảng 1 cho cấu hình chi tiết): <https://arxiv.org/abs/1409.1556>

Đóng góp của VGGNet

Insight cần đạt

Ba đóng góp lâu dài của VGGNet:

1. **Kernel 3×3 là tiêu chuẩn:** Sau VGG, không ai dùng kernel lớn hơn 3×3 ở các conv layer (trừ layer đầu trong một số kiến trúc đặc biệt).

2. **Pattern “giảm spatial, tăng channels”:** Đây là design principle phổ quát — ResNet, DenseNet, EfficientNet đều tuân thủ.
3. **Transfer learning với VGG features:** Trước khi ResNet ra đời, *features từ VGG-16 pretrained trên ImageNet* là điểm xuất phát cho hầu hết task vision. Đến nay, một số bài toán y khoa vẫn dùng VGG-16 vì đơn giản và pretrained features rất tốt.

Hạn chế của VGGNet

Lưu ý quan trọng

Dù đẹp về mặt thiết kế, VGGNet bộc lộ ngay những hạn chế *định hướng cho thế hệ sau*:

- **Quá nhiều tham số:** VGG-16 có **138 triệu tham số**, trong đó $\sim 103M$ chỉ ở FC layer đầu tiên (Flatten 25088 $\rightarrow$ 4096). Đây là $\sim 75\%$ tham số của toàn mạng! $\Rightarrow$ ResNet và sau này GoogLeNet/Inception thay FC khổng lồ bằng **Global Average Pooling** để khắc phục.
- **Tốn bộ nhớ:** File checkpoint VGG-16 nặng ~ 528 MB — không phù hợp triển khai trên thiết bị di động.
- **Tính toán đắt:** forward pass tốn nhiều FLOPs hơn ResNet-50 mặc dù ít layer hơn.
- **Vẫn không train được sâu hơn:** VGG-19 đã là tới hạn. Thử VGG-30, VGG-50 thì *accuracy giảm* — chính là *degradation problem* mà ResNet sẽ giải quyết.
- **Không có normalization hiện đại:** VGG ra đời *trước* BatchNorm (2015) — nên rất khó train, phải dùng tricks như khởi tạo cẩn thận và pretrain từng phần.

1.3 So sánh AlexNet và VGGNet — Bảng tổng hợp

Tiêu chí	AlexNet (2012)	VGGNet (2014)
Số layer học được	8 (5 conv + 3 FC)	16–19 (13–16 conv + 3 FC)
Kernel size	11×11 , 5×5 , 3×3	Chỉ 3×3
Số tham số	60M	138M (VGG-16)
Top-5 error (ImageNet)	15.3%	7.3%
Triết lý thiết kế	“Sâu hơn LeNet, dùng GPU”	“Càng sâu càng tốt, kernel nhỏ”
Đóng góp lớn nhất	ReLU + GPU + Dropout	Triết lý 3×3 , design pattern
Hạn chế chính	Không scale được sâu hơn	FC khổng lồ, vẫn không train được rất sâu

1.4 Hai nỗi đau lớn còn lại sau VGGNet

Insight cần đạt

Đến cuối 2014, cộng đồng đã có hai bài học rõ ràng:

1. **Càng sâu càng tốt** (từ AlexNet 8 layer → VGG-19 19 layer, accuracy luôn tăng).
2. **Kernel 3×3 là chuẩn** (từ VGG).

Nhưng **hai nỗi đau lớn** vẫn còn:

1. **Không thể train được mạng *thực sự* sâu** (>20 layer kiểu VGG): degradation problem xuất hiện.
2. **Quá nhiều tham số ở FC** (75% của VGG-16): tốn memory, dễ overfit.

Đây chính là những vấn đề mà thế hệ tiếp theo — bắt đầu từ ResNet — sẽ giải quyết. Chúng ta cùng đi vào phần đó ở section sau.

2. Bức tranh toàn cảnh: Tại sao cần kiến trúc mới?

Lưu ý quan trọng

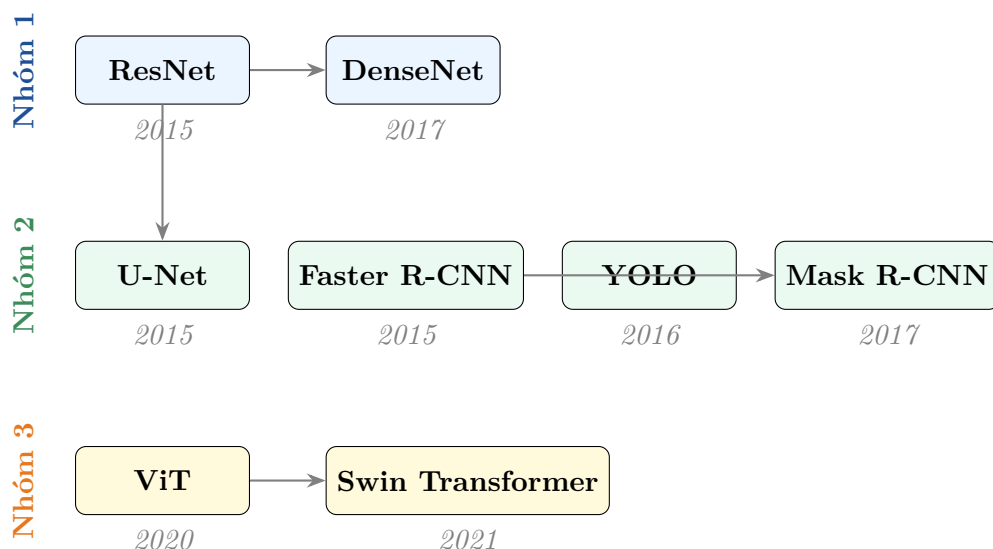
Sinh viên cần hiểu được rằng *mỗi kiến trúc mới không phải ngẫu nhiên ra đời* mà nó giải quyết được nhược điểm của kiến trúc trước đó.

2.1 Ba hạn chế lớn của CNN cổ điển

Sau giai đoạn “vàng son” của VGG (2014), cộng đồng Computer Vision nhận ra rằng CNN cổ điển có **ba hạn chế lớn** mà việc cải tiến theo lối cũ (stack thêm layer, tăng channel) không thể giải quyết được:

Hạn chế của CNN cơ bản	Hướng giải quyết	Đại diện
Khó train mạng rất sâu (vanishing gradient)	Skip connection / Residual learning	ResNet, DenseNet
Receptive field cục bộ, không nhìn xa được	Tăng vùng nhìn / Attention	Vision Transformer (ViT)
Chỉ phân loại được, không xử lý được vị trí/pixel	Kiến trúc encoder-decoder + đa nhiệm	U-Net, Faster R-CNN, YOLO, Mask R-CNN

2.2 Dòng chảy lịch sử của 8 kiến trúc



3. Nhóm 1 — Cải tiến chính CNN (2014–2017)

3.1 ResNet (2015) — Cuộc cách mạng “skip connection”

Vấn đề trước đó

Câu hỏi

Năm 2014, người ta tin rằng *càng sâu càng tốt*. Vậy tại sao khi xếp chồng 30 layer trở lên, mạng lại **tệ hơn** mạng 20 layer? Đây là do *overfit* chăng?

Không phải overfit. Khi tăng độ sâu, không chỉ *test error* mà cả *training error* cũng **tăng lên** — điều này không thể giải thích bằng overfit. Hiện tượng này được gọi là **degradation problem**.

Nguyên nhân thực sự: vanishing gradient. Khi gradient lan ngược qua nhiều layer, mỗi lần nhân với một ma trận trọng số nhỏ hơn 1 (qua đạo hàm của activation), gradient *teo nhỏ theo cấp số nhân*, khiến các layer đầu tiên gần như không học được gì.

Ý tưởng cốt lõi — Residual Learning

Khái niệm cốt lõi

Thay vì học trực tiếp hàm mục tiêu $\mathcal{H}(x)$, hãy học **phần dư** (residual):

$$\mathcal{F}(x) = \mathcal{H}(x) - x \quad \Longleftrightarrow \quad \mathcal{H}(x) = \mathcal{F}(x) + x$$

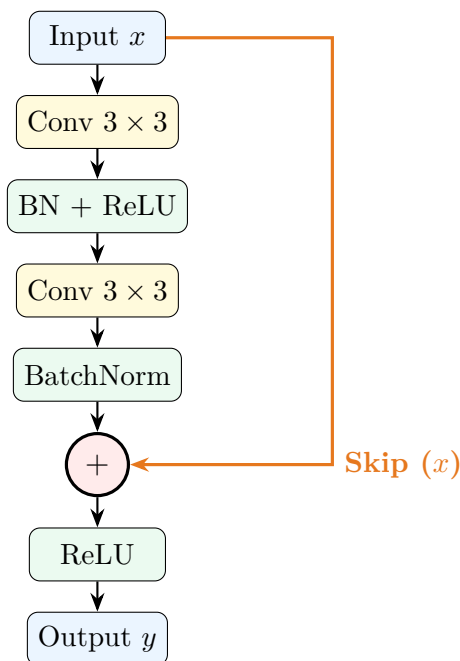
Đầu ra của một block là:

$$y = \mathcal{F}(x; \theta) + x$$

Phần “+ x ” chính là **skip connection** (hay *shortcut connection*) — đường tắt cho cả forward signal và backward gradient.

Minh họa Residual Block

Xem bản đầy đủ cho ResNet https://oi.readthedocs.io/en/latest/_images/cnn_vs_resnet_vs_densenet.png



Giải bài toán nào?

Ứng dụng thực tế

- **Phân loại ảnh tổng quát:** ResNet-50, ResNet-101 trở thành *backbone mặc định* cho hầu hết task vision.
- **Transfer learning:** ResNet pretrained trên ImageNet là điểm xuất phát phổ biến nhất cho mọi bài toán xử lý ảnh thực tế — từ phân loại bệnh lý y khoa đến nhận dạng sản phẩm.
- **Backbone cho detection/segmentation:** Faster R-CNN, Mask R-CNN, YOLO — tất cả đều dùng ResNet làm backbone.

Vì sao thắng?

Insight cần đạt

Bốn lý do then chốt:

1. **Gradient flow tốt:** Skip connection cho phép gradient đi tắt qua nhiều layer, không bị vanish.
2. **Optimization dễ hơn:** Mạng có thể dễ dàng học *identity mapping* bằng cách đẩy $\mathcal{F}(x) \rightarrow 0$.
3. **Train được mạng 152 layer:** ResNet-152 đạt sai số thấp hơn cả VGG-19 và ít tham số hơn.
4. **Ảnh hưởng kéo dài:** Skip connection trở thành “viên gạch nền” của hầu hết kiến trúc sau này — kể cả Transformer.

3.2 DenseNet (2017) — Cực đoan hóa skip connection

Vấn đề và Ý tưởng

Câu hỏi

Nếu skip connection của ResNet (kết nối layer i với layer $i + 2$) hữu ích như vậy, thì tại sao không kết nối tất cả các layer với nhau?

Đây chính là ý tưởng của DenseNet: mỗi layer nhận đầu vào là concatenation của tất cả feature maps từ mọi layer trước đó.

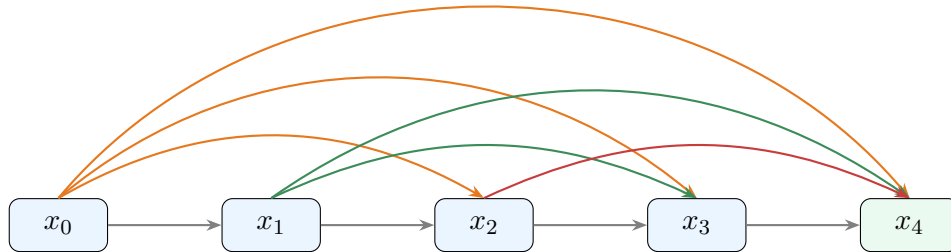
Khái niệm cốt lõi

So sánh hai cơ chế kết nối:

- **ResNet:** $x_\ell = \mathcal{F}_\ell(x_{\ell-1}) + x_{\ell-1}$ — cộng.

- **DenseNet:** $x_\ell = \mathcal{F}_\ell([x_0, x_1, \dots, x_{\ell-1}])$ — nối (concatenation).

Minh họa Dense Block



Mỗi layer kết nối tới **TẤT CẢ** layer sau

Ví dụ minh họa

Xem hình minh họa kiến trúc DenseNet:

- Sơ đồ học thuật trên ResearchGate: https://www.researchgate.net/figure/DenseNet-Architecture-Because-each-layer-of-a-DenseNet-architecture-is-fig1_378536444
- Hướng dẫn tổng quát và tổng hợp mã nguồn: <https://paperswithcode.com/method/densenet>
- Paper gốc (tham khảo Hình 1 và Bảng 1 cho cấu hình chi tiết): <https://arxiv.org/abs/1608.06993>

Giải bài toán nào?

Ứng dụng thực tế

- **Bài toán có ít data:** DenseNet tận dụng feature ở mọi cấp độ, giảm tham số, ít overfit.
- **Ảnh y khoa:** CT, MRI, X-ray — nơi data thường ít và đắt.
- **Ảnh vệ tinh:** Tương tự, dataset thường nhỏ.

So sánh ResNet và DenseNet

Tiêu chí	ResNet	DenseNet
Cơ chế kết nối	Cộng (addition)	Nối (concatenation)
Số tham số	Nhiều hơn	Ít hơn (tái sử dụng feature)
Memory	Thấp hơn	Cao hơn (lưu nhiều feature map)
Hiệu quả với data ít	Tốt	Tốt hơn
Phổ biến	Backbone mặc định	Niche (medical, satellite)

Ghi chú

Trong thực tế, ResNet vẫn được dùng nhiều hơn DenseNet vì **memory footprint thấp hơn**. DenseNet mạnh khi data nhỏ, nhưng với big data, ResNet và các biến thể (ResNeXt, Wide ResNet) thường thắng.

4. Nhóm 2 — Vượt qua phân loại: Detection & Segmentation

4.1 Ba câu hỏi của Computer Vision

CNN cổ điển chỉ trả lời được câu hỏi “*Đây là cái gì?*”. Nhưng trong thực tế, ta cần trả lời thêm:

Câu hỏi	Bài toán	Kiến trúc tiêu biểu
“Đây là cái gì?”	Classification	ResNet, DenseNet
“Cái đó ở đâu trong ảnh?”	Object Detection	Faster R-CNN, YOLO
“Mỗi pixel thuộc về cái gì?”	Semantic Segmentation	U-Net, DeepLab
“Có bao nhiêu cá thể, biên giới từng cá thể?”	Instance Segmentation	Mask R-CNN

4.2 U-Net (2015) — Vua của Segmentation

Vấn đề trước đó

Câu hỏi

CNN classification thông thường *nén ảnh* qua nhiều pooling layer cho đến khi còn 1×1 . Nhưng nếu ta cần xuất ra *một mask cùng kích thước ảnh gốc* (ví dụ: phân vùng khối u trong ảnh CT), làm sao phục hồi spatial resolution đã mất?

Ý tưởng cốt lõi — Encoder-Decoder + Skip

Ví dụ minh họa

Xem hình minh họa kiến trúc U-Net:

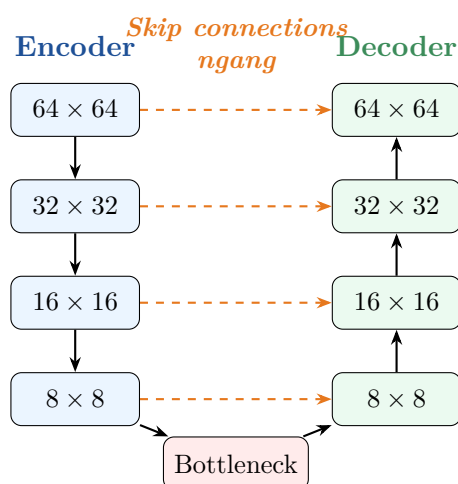
- Sơ đồ học thuật trên ResearchGate: https://www.researchgate.net/figure/Illustration-of-the-U-net-architecture-The-figure-illustrates-the-U-net-fig2_331406702
- Hướng dẫn tổng quát kèm phân tích encoder-decoder và skip connections: <https://medium.com/@alejandro.itoaramendia/decoding-the-u-net-a-complete-guide-810b1c6d56d8>
- Paper gốc (Ronneberger et al., 2015 — tham khảo Hình 1 cho cấu hình chi tiết): <https://arxiv.org/abs/1505.04597>

Khái niệm cốt lõi

U-Net có hình chữ U:

- Bên trái (Encoder / Contracting path):** Nén dần ảnh qua các conv + pooling, giống CNN bình thường $\Rightarrow$ học *ngữ cảnh* (context).
- Bên phải (Decoder / Expanding path):** Phục hồi dần kích thước qua *upsampling* $\Rightarrow$ học *vị trí* (localization).
- Skip connections ngang:** Từ encoder sang decoder ở cùng độ phân giải, giúp giữ chi tiết spatial.

Minh họa kiến trúc U-Net



Giải bài toán nào?

Ứng dụng thực tế

- **Segmentation ảnh y khoa:** khối u, mạch máu, cơ quan trong CT/MRI — U-Net ra đời chính cho bài toán này (segmentation tế bào trong ảnh hiển vi điện tử).
- **Ảnh vệ tinh:** phân vùng đất, rừng, nước, công trình.
- **Xe tự lái:** phân vùng đường, vạch kẻ, lề đường.
- **Phân tích ảnh sinh học:** đếm và phân vùng tế bào, hạt giống.

Vì sao thắng?

Insight cần đạt

- **Skip connection ngang giữ chi tiết pixel-level:** segmentation cần biết chính xác từng pixel thuộc lớp nào — mất chi tiết là sai ngay.
- **Encoder học context, decoder học localization:** kết hợp hai loại thông tin.
- **Hiệu quả với data ít:** U-Net được thiết kế cho dataset y khoa nhỏ.
- **Trở thành “mẫu chuẩn”:** kiến trúc encoder-decoder + skip connection được tái sử dụng trong rất nhiều mạng sau này (Stable Diffusion cũng dùng U-Net!).

4.3 Mô hình R-CNN (Regions with CNN features)

1. Vấn đề giải quyết

Trước khi R-CNN ra đời, các phương pháp phát hiện đối tượng (Object Detection) truyền thống phụ thuộc chủ yếu vào kỹ thuật cửa sổ trượt (sliding window) kết hợp với các bộ rút trích đặc trưng thủ công như HOG (Histogram of Oriented Gradients) hoặc SIFT. Các phương pháp này gặp hạn chế lớn về độ chính xác, khả năng khái quát hóa kém và gặp khó khăn trong việc xác định chính xác vị trí của các đối tượng có kích thước và tỉ lệ đa dạng. R-CNN ra đời nhằm nâng cao đột phá độ chính xác bằng cách lần đầu tiên ứng dụng thành công mạng thần kinh tích chập (CNN) sâu vào bài toán phát hiện đối tượng.

2. Ý tưởng cốt lõi

Ý tưởng chính của R-CNN là chuyển bài toán phát hiện đối tượng thành bài toán phân loại vùng ảnh thông qua hệ thống gồm 4 bước độc lập:

- **Đề xuất vùng (Region Proposals):** Sử dụng thuật toán *Selective Search* để quét qua hình ảnh đầu vào và trích xuất ra khoảng 2000 vùng chứa đối tượng tiềm năng độc lập với các lớp (class-independent).

- **Trích xuất đặc trưng (Feature Extraction):** Biến đổi hình học (warp) kích thước của từng vùng đề xuất về một kích cỡ cố định (ví dụ: 227×227 pixel) rồi đưa qua một mạng CNN tiền huấn luyện (như AlexNet) để trích xuất ra một vector đặc trưng kích thước 4096 chiều.
- **Phân loại (Classification):** Sử dụng các bộ phân loại SVM tuyến tính (Linear SVM) đã được huấn luyện riêng cho từng lớp đối tượng để chấm điểm và xác định xem vùng đó chứa đối tượng nào.
- **Tính chỉnh vị trí (Bounding-box Regression):** Áp dụng một mô hình hồi quy tuyến tính để điều chỉnh và tối ưu lại tọa độ của hộp bao (bounding box) sao cho khớp khít nhất với thực tế.

3. Ứng dụng

R-CNN được áp dụng trong các bài toán Thị giác máy tính liên quan đến phát hiện, định vị và phân loại đa đối tượng (Multi-object Detection and Localization) trong ảnh tĩnh đối với các tập dữ liệu như PASCAL VOC hoặc ImageNet.

4. Kiến trúc minh họa

Bạn có thể tham khảo sơ đồ kiến trúc nguyên bản của R-CNN thông qua liên kết sau:

<https://medium.com/@mustapha.aitigunaoun/evolution-of-object-detection-r-cnn-to-fa>

5. Nhược điểm và hạn chế

R-CNN gặp thất bại lớn trong các bài toán yêu cầu xử lý thời gian thực (Real-time applications) do các nhược điểm nghiêm trọng sau:

- **Tốc độ xử lý cực kỳ chậm:** Việc phải chạy mạng CNN độc lập cho toàn bộ 2000 vùng đề xuất trên mỗi bức ảnh gây ra sự lãng phí tài nguyên tính toán khổng lồ. Quá trình kiểm thử (inference) mất tới gần 47 giây cho một bức ảnh trên GPU.
- **Tốn không gian lưu trữ:** Do quá trình huấn luyện rời rạc, các đặc trưng trích xuất từ 2000 vùng của mọi bức ảnh trong tập dữ liệu phải được lưu cứng vào đĩa trước khi huấn luyện SVM, đòi hỏi hàng trăm GB dung lượng lưu trữ.
- **Huấn luyện phức tạp (Multi-stage pipeline):** R-CNN không phải là một mạng end-to-end. Việc huấn luyện phải chia làm 3 giai đoạn riêng biệt (CNN $\rightarrow$ SVM $\rightarrow$ Bounding-box Regressor), khiến hệ thống không thể tối ưu hóa toàn cục bằng lan truyền ngược.

4.4 Mô hình Fast R-CNN

1. Vấn đề giải quyết

Fast R-CNN được đề xuất bởi chính tác giả của R-CNN nhằm giải quyết triệt để các điểm nghẽn về tốc độ tính toán, dung lượng lưu trữ bộ nhớ và sự phức tạp trong quy trình huấn luyện đa giai đoạn của phiên bản R-CNN tiền nhiệm.

2. Ý tưởng cốt lõi

Thay vì trích xuất đặc trưng của từng vùng đề xuất một cách riêng lẻ và lặp đi lặp lại, Fast R-CNN đã gộp toàn bộ quá trình thành một mạng thống nhất nhờ vào các cải tiến:

- **Một lần trích xuất đặc trưng (Single Forward Pass):** Đưa toàn bộ bức ảnh gốc qua mạng tích chập CNN *đúng một lần* duy nhất để thu được một bản đồ đặc trưng (Feature Map) chung cho toàn bộ bức ảnh.
- **Lớp RoI Pooling (Region of Interest Pooling):** Các vùng đề xuất (vẫn lấy từ Selective Search) sẽ được chiếu (project) trực tiếp lên bản đồ đặc trưng chung. Lớp RoI Pooling có nhiệm vụ trích xuất và biến đổi các vùng đặc trưng này thành các vector có kích thước cố định, bất kể kích thước vùng đề xuất ban đầu là bao nhiêu.
- **Mạng đa nhiệm tích hợp (Multi-task Loss):** Vector đặc trưng sau RoI Pooling được đưa qua các lớp liên kết đầy đủ (FC layers), sau đó rẽ nhánh song song ra hai đầu ra: một đầu sử dụng hàm *Softmax* để phân loại xác suất đối tượng, một đầu hồi quy để tính chỉnh tọa độ bounding box. Toàn bộ mạng được huấn luyện đồng thời (end-to-end) bằng một hàm mất mát đa nhiệm phối hợp (Multi-task loss).

3. Ứng dụng

Tương tự như R-CNN, mô hình được dùng cho bài toán phát hiện và định vị đối tượng nhưng được tối ưu hóa mạnh mẽ để ứng dụng vào các hệ thống đòi hỏi hiệu năng cao và phản hồi nhanh hơn.

4. Kiến trúc minh họa

Bạn có thể tham khảo sơ đồ kiến trúc cải tiến của Fast R-CNN thông qua liên kết sau:

<https://lilianweng.github.io/posts/2017-12-31-object-recognition-part-1/fast-rcnn.png>

5. Nhược điểm và hạn chế

Mặc dù tốc độ huấn luyện nhanh hơn R-CNN gấp 9 lần và tốc độ kiểm thử nhanh hơn tới 150 lần, Fast R-CNN vẫn chưa thể coi là một kiến trúc chạy thời gian thực hoàn chỉnh (Real-time bottleneck) vì lý do sau:

- **Phụ thuộc vào thuật toán đề xuất eksternal:** Mạng Fast R-CNN dù chạy rất nhanh trên GPU nhưng nó vẫn bắt buộc phải sử dụng thuật toán *Selective Search* (chạy trên CPU) để tìm kiếm các vùng đề xuất ban đầu. Quá trình tính toán vùng đề xuất này mất khoảng 2 giây cho mỗi bức ảnh, tạo thành một “nút thắt cổ chai” chí mạng khiến hệ thống tổng thể không thể đạt tốc độ khung hình cao phục vụ

cho bài toán xử lý video trực tiếp (bài toán này sau đó mới được giải quyết triệt để bởi cấu trúc *Faster R-CNN* nhờ mạng RPN).

4.5 Faster R-CNN (2015) — Detection chính xác cao

Vấn đề trước đó

Ví dụ minh họa

Xem hình minh họa kiến trúc Faster R-CNN:

- Sơ đồ học thuật trên ResearchGate (backbone + RPN + Fast R-CNN head): https://www.researchgate.net/figure/The-Faster-R-CNN-architecture-an-input-image-is-fed-through-a-backbone-CNN-to-fig1_358847353
- Hướng dẫn tổng quát kèm phân tích RPN, anchors và RoI Pooling: <https://www.digitalocean.com/community/tutorials/faster-r-cnn-explained-object-detection>
- Paper gốc (Ren et al., 2015 — tham khảo Hình 2 cho cấu hình chi tiết): <https://arxiv.org/abs/1506.01497>

Câu hỏi

Làm sao trả lời được câu hỏi “*Có cái gì ở đâu?*”, tức là vừa nhận dạng object, vừa vẽ **bounding box** bao quanh nó?

Trước Faster R-CNN, các phương pháp như R-CNN và Fast R-CNN dùng thuật toán cổ điển (*Selective Search*) để đề xuất hàng nghìn vùng có thể chứa object — rất chậm.

Ý tưởng cốt lõi — Two-Stage Detection

Khái niệm cốt lõi

Faster R-CNN gồm **hai** giai đoạn:

1. **Region Proposal Network (RPN):** một CNN nhỏ duyệt feature map, đề xuất các *anchor box* có khả năng chứa object.
2. **Classification + Box Regression Head:** cho mỗi proposal, mạng phân loại class và tính chỉnh tọa độ bounding box.

Cả hai giai đoạn dùng *chung* một backbone CNN $\Rightarrow$ nhanh hơn rất nhiều so với R-CNN.

Giải bài toán nào?

Ứng dụng thực tế

- **Y khoa:** phát hiện khối u trong ảnh X-quang, CT — cần độ chính xác cao, không cần real-time.
- **Kiểm tra chất lượng công nghiệp:** phát hiện lỗi sản phẩm trong dây chuyền.
- **Phân tích ảnh vệ tinh:** đếm tàu, máy bay, công trình.

Vì sao thắng (và vì sao có giới hạn)?

Insight cần đạt

Ưu điểm:

- Two-stage cho phép tinh chỉnh nhiều bước $\Rightarrow$ **độ chính xác cao**.
- Học end-to-end (cả RPN và classifier).

Hạn chế:

- **Chậm:** 5–7 FPS trên GPU — không real-time.
- Phức tạp, khó triển khai.

4.6 YOLO (2016, hiện đã đến v8/v11) — Detection Real-time

Ví dụ minh họa

Xem hình minh họa kiến trúc YOLO:

- Sơ đồ học thuật trên ResearchGate (24 conv layers + 2 fully connected): https://www.researchgate.net/figure/Architecture-of-YOLO-a-single-shot-object-detection-model-21_fig2_338760535
- Hướng dẫn tổng quát kèm các phiên bản YOLOv1/v2/v3/.../v8 trở lên: <https://www.datacamp.com/blog/yolo-object-detection-explained>
- Paper gốc (Redmon et al., 2016 — tham khảo Hình 3 cho cấu hình chi tiết): <https://arxiv.org/abs/1506.02640>

Vấn đề và Ý tưởng

Câu hỏi

Trong các ứng dụng real-time như xe tự lái hay camera giám sát, Faster R-CNN *quá chậm*. Làm sao detect được object với tốc độ > 30 FPS?

Khái niệm cốt lõi

YOLO — You Only Look Once: Bỏ hoàn toàn giai đoạn region proposal. Chia ảnh thành lưới $S \times S$, mỗi ô lưới *đồng thời dự đoán*:

- Tọa độ bounding box (x, y, w, h) .
- Confidence score (có object hay không).
- Class probabilities.

Tất cả trong **một lần forward duy nhất**.

Giải bài toán nào?

Ứng dụng thực tế

Detection real-time:

- **Camera giám sát:** phát hiện vi phạm, đếm người vào ra cửa hàng.
- **Xe tự lái:** phát hiện người đi bộ, xe khác, biển báo — cần phản ứng tức thời.
- **Drone, robot:** phát hiện vật cản, theo dõi mục tiêu.
- **Giao thông thông minh:** phát hiện vi phạm tốc độ, đếm xe.
- **Thể thao:** theo dõi vận động viên, bóng, trang phục.

Trade-off giữa Faster R-CNN và YOLO

Tiêu chí	Faster R-CNN	YOLO
Số giai đoạn	Two-stage	One-stage
Tốc độ	5–7 FPS	30–150+ FPS
Độ chính xác	Cao hơn	Thấp hơn một chút
Real-time	Không	Có
Phù hợp với	Y khoa, công nghiệp	Xe tự lái, drone, giám sát

Ghi chú

YOLO không phải “thay thế” Faster R-CNN — mỗi mạng phù hợp với một loại bài toán. **Quy tắc chọn:** cần real-time $\Rightarrow$ YOLO; cần accuracy cao nhất, có thời

gian xử lý $\Rightarrow$ Faster R-CNN.

4.7 Mask R-CNN (2017) — Instance Segmentation

Ví dụ minh họa

Xem hình minh họa kiến trúc Mask R-CNN:

- Sơ đồ học thuật trên ResearchGate (backbone + RPN + RoIAlign + mask branch): https://www.researchgate.net/figure/Overall-architecture-of-a-Mask-R-CNN_fig5_346075907
- Hướng dẫn tổng quát kèm phân tích RoIAlign, FPN và mask head: <https://blog.roboflow.com/mask-rcnn/>
- Paper gốc (He et al., 2017 — tham khảo Hình 1 và Hình 3 cho cấu hình chi tiết): <https://arxiv.org/abs/1703.06870>

Vấn đề và Ý tưởng

Câu hỏi

Semantic segmentation (như U-Net) cho biết mỗi pixel thuộc *class* nào, nhưng không phân biệt được các *cá thể* cùng class. Nếu trong ảnh có 5 con mèo, U-Net chỉ cho ra một mặt nạ “mèo” — không biết đâu là mèo 1, đâu là mèo 2. Làm sao vừa detect từng cá thể, vừa vẽ mask chính xác cho *từng* cá thể?

Khái niệm cốt lõi

Mask R-CNN = Faster R-CNN + thêm một “đầu” (head) mask.
Bên cạnh hai đầu của Faster R-CNN (classification và bounding box regression), thêm một đầu thứ ba: *mask head* — một mạng nhỏ vẽ binary mask cho từng object đã detect.

Giải bài toán nào?

Ứng dụng thực tế

- **Đếm tế bào:** trong ảnh kính hiển vi — vừa phân biệt từng tế bào, vừa biết hình dạng chính xác.
- **Đếm và theo dõi xe:** trên đường — phân biệt từng xe.
- **Phân biệt từng con vật trong đàn:** đếm gia súc, chim, cá.
- **Robot gắp vật (robotic grasping):** cần biết chính xác hình dạng từng vật.
- **AR / Photo editing:** cắt từng object riêng biệt để chỉnh sửa.

Phân biệt 3 loại segmentation

Loại	Mô tả	Mạng tiêu biểu
Semantic segmentation	Phân vùng theo class, không phân biệt cá thể	U-Net, DeepLab
Instance segmentation	Phân vùng từng cá thể riêng	Mask R-CNN
Panoptic segmentation	Cả hai cùng lúc (background + instances)	Panoptic FPN

5. Nhóm 3 — Cuộc cách mạng Transformer trong Vision (2020+)

5.1 Vision Transformer (ViT, 2020) — CNN không còn là duy nhất

Vấn đề trước đó

Câu hỏi

CNN bị giới hạn bởi **receptive field cục bộ**: muốn “nhìn” toàn ảnh phải stack rất nhiều layer. Transformer trong NLP đã chứng minh self-attention rất mạnh ở việc nhìn ngữ cảnh xa. **Liệu có thể áp dụng Transformer thuần cho ảnh không?**

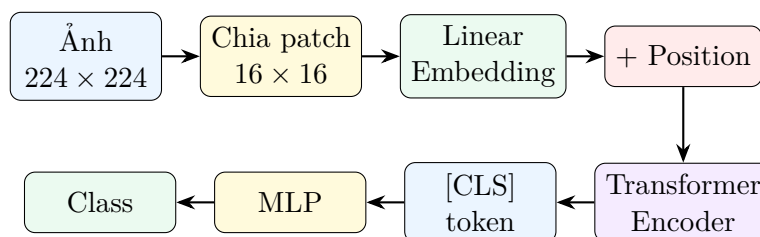
Ý tưởng cốt lõi — Patch như Word

Khái niệm cốt lõi

ViT bỏ hoàn toàn convolution. Thay vào đó:

1. Cắt ảnh thành các **patch** cỡ 16×16 pixel.
2. Mỗi patch được flatten thành vector và embed như *một “từ”*.
3. Thêm *positional embedding* (vì Transformer không có thứ tự).
4. Đưa chuỗi patch vào **Transformer Encoder** chuẩn (multi-head self-attention).
5. Lấy embedding của token đặc biệt [CLS] đưa qua MLP để phân loại.

Minh họa pipeline ViT



Giải bài toán nào?

Ứng dụng thực tế

- **Phân loại ảnh với big data:** khi có ≥ 10 triệu ảnh (JFT-300M), ViT **thắng** CNN.
- **Pretraining quy mô lớn:** CLIP, DINO, MAE đều dùng ViT làm backbone.
- **Hiểu ngữ cảnh toàn cục:** nhờ self-attention, mỗi patch “nhìn thấy” mọi patch khác *từ layer đầu tiên*.

Vì sao ViT thắng CNN khi đủ data?

Insight cần đạt

Câu chuyện về inductive bias:

- **CNN có inductive bias mạnh:** giả định ảnh có cấu trúc *cục bộ* (locality) và *translation invariance*. Bias này *rất hữu ích* khi data ít — model không cần học các tính chất này từ data.
- **ViT không có những bias đó:** cần học mọi thứ *từ data*. Khi data ít, ViT thua. Nhưng khi đủ data, ViT *linh hoạt hơn* — có thể học các pattern mà CNN không thể biểu diễn được.

Hạn chế của ViT

Lưu ý quan trọng

- **Cần $\geq 10M$ ảnh** để vượt CNN. Với dataset nhỏ, CNN vẫn tốt hơn.
- **Compute đắt:** self-attention có độ phức tạp $O(n^2)$ với n là số patch — không scale tốt với ảnh độ phân giải cao.
- **Hard to train:** cần các kỹ thuật như learning rate warmup, large batch.

5.2 Swin Transformer (2021) — Transformer “biết tiết kiệm”

Vấn đề của ViT

Câu hỏi

Self-attention của ViT có độ phức tạp $O(n^2)$ với n là số patch. Với ảnh 1024×1024 và patch 16×16 , ta có $n = 4096$ patches $\Rightarrow n^2 \approx 17$ triệu phép tính attention cho mỗi layer. **Quá đắt cho ảnh độ phân giải cao!**

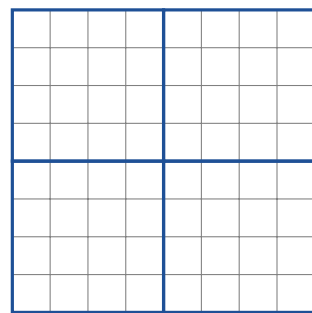
Ý tưởng cốt lõi — Windowed Attention + Shift

Khái niệm cốt lõi

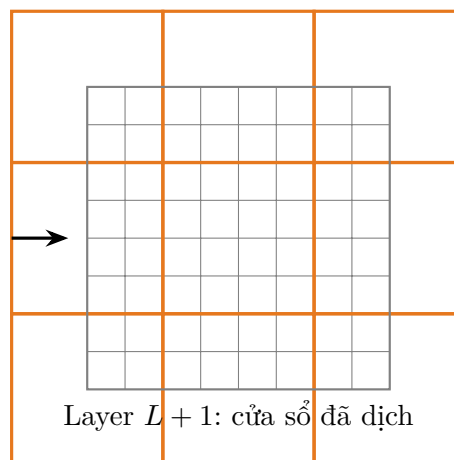
Swin Transformer (*Shifted Windows*) chỉ tính attention trong các **cửa sổ nhỏ** (ví dụ 7×7 patches), không phải toàn ảnh:

1. Chia patches thành các *cửa sổ* không chồng lấp.
2. Tính self-attention *trong* mỗi cửa sổ $\Rightarrow$ chi phí giảm từ $O(n^2)$ xuống $O(n)$.
3. Ở layer kế tiếp, **dịch cửa sổ** (shift) đi nửa size $\Rightarrow$ các vùng khác nhau “nói chuyện” với nhau.

Minh họa Shifted Window



Layer L : 4 cửa sổ chuẩn



Layer $L + 1$: cửa sổ đã dịch

Giải bài toán nào?

Ứng dụng thực tế

- **Detection và segmentation ảnh độ phân giải cao:** thay thế ResNet làm backbone trong COCO, ADE20K.
- **Ảnh y khoa độ phân giải cao:** CT, MRI, pathology slides.
- **Ảnh vệ tinh chi tiết:** phân vùng tòa nhà, đường xá, cây cối.
- **Backbone đa nhiệm:** dùng được cho classification, detection, segmentation

cùng lúc — “general-purpose vision backbone”.

Vì sao Swin tốt hơn ViT cho nhiều bài toán?

Insight cần đạt

- **Tuyến tính theo kích thước ảnh:** $O(n)$ thay vì $O(n^2) \Rightarrow$ scale được lên ảnh lớn.
- **Multi-scale feature maps:** giống CNN, có hierarchy of resolution $\Rightarrow$ dễ tích hợp với detection/segmentation head.
- **Vẫn có global context:** qua các shifted windows ở các layer kế tiếp.
- **Cần ít data hơn ViT thuần** để đạt accuracy tốt — vì có inductive bias cục bộ.

6. Tổng hợp: Bảng so sánh 8 kiến trúc

Mạng	Năm	Đóng góp chính	Bài toán giỏi nhất
ResNet	2015	Skip connection cho mạng sâu	Classification, transfer learning
DenseNet	2017	Concatenation thay vì addition	Medical, satellite (data ít)
U-Net	2015	Encoder-decoder + skip ngang	Semantic segmentation (y khoa)
Faster R-CNN	2015	Region Proposal Network	Detection accuracy cao
YOLO	2016	One-stage detection	Real-time detection
Mask R-CNN	2017	Detection + instance mask	Instance segmentation
ViT	2020	Patch như word, attention thuần	Classification với big data
Swin	2021	Windowed attention + shift	Detection/segmentation HD

6.1 Khi nào dùng kiến trúc nào? — Cây quyết định

Khái niệm cốt lõi

Bài toán của bạn là gì?

- **Phân loại ảnh đơn giản:**
 - Data ít (< 100K ảnh) $\Rightarrow$ **ResNet-50** (transfer learning).
 - Data nhiều (> 10M ảnh) $\Rightarrow$ **ViT**.
 - Data y khoa, vệ tinh, rất ít $\Rightarrow$ **DenseNet**.
- **Phát hiện object (có bounding box):**
 - Cần real-time $\Rightarrow$ **YOLO**.
 - Cần accuracy cao, không cần real-time $\Rightarrow$ **Faster R-CNN**.
- **Phân vùng pixel-level:**
 - Không cần phân biệt cá thể (semantic) $\Rightarrow$ **U-Net**.
 - Cần phân biệt từng cá thể (instance) $\Rightarrow$ **Mask R-CNN**.
- **Ảnh độ phân giải cao:**
 - $\Rightarrow$ **Swin Transformer** làm backbone.

6.2 Bài học triết lý quan trọng nhất

Insight cần đạt

Bốn bài học cốt lõi từ 8 kiến trúc trên:

1. **Skip connection** là một trong những phát kiến quan trọng nhất của Deep Learning. Nó xuất hiện trong ResNet, DenseNet, U-Net, và cả Transformer.
2. Mỗi kiến trúc giải quyết một vấn đề cụ thể. Không có “best model” — chỉ có “best model cho bài toán cụ thể”.
3. **Inductive bias** là con dao hai lưỡi. CNN có bias mạnh $\Rightarrow$ tốt với data ít, kém với data nhiều. ViT ngược lại.
4. **Encoder-decoder** là pattern phổ quát. Từ U-Net (vision), đến seq2seq (NLP), đến diffusion models hiện đại.

7. Code minh họa: Sử dụng các mô hình pretrained

Phần lớn các kiến trúc trên đã có sẵn trong `torchvision` và `HuggingFace`. Sinh viên hiếm khi phải implement từ đầu — thay vào đó, ta dùng pretrained model.

Listing 1: ResNet pretrained cho transfer learning

```

1 import torch
2 import torchvision.models as models
3
4 # Load ResNet-50 pretrained
5 model = models.resnet50(weights='IMAGENET1K_V2')
6
7 # Thay layer cuoi cho task cua ban (gia su 10 class)
8 import torch.nn as nn
9 model.fc = nn.Linear(model.fc.in_features, 10)
10
11 # Dong cung (freeze) backbone, chi train layer cuoi
12 for param in model.parameters():
13     param.requires_grad = False
14 for param in model.fc.parameters():
15     param.requires_grad = True

```

Listing 2: U-Net cho segmentation (segmentation_models_pytorch)

```

1 import segmentation_models_pytorch as smp
2
3 # U-Net voi encoder ResNet-50 pretrained
4 model = smp.Unet(
5     encoder_name="resnet50",
6     encoder_weights="imagenet",
7     in_channels=3,
8     classes=1 # nhi phan: pixel thuoc khoi u hay khong
9 )

```

Listing 3: YOLO v8 cho detection real-time

```

1 from ultralytics import YOLO
2
3 # Load YOLOv8 pretrained tren COCO
4 model = YOLO('yolov8n.pt')
5
6 # Inference tren anh
7 results = model('path/to/image.jpg')
8
9 # Train tren dataset cua ban
10 model.train(data='custom.yaml', epochs=50, imgsz=640)

```

Listing 4: Vision Transformer pretrained

```

1 from transformers import ViTForImageClassification, ViTImageProcessor
2
3 # Load ViT-Base pretrained tren ImageNet-21k
4 processor = ViTImageProcessor.from_pretrained('google/vit-base-
5     patch16-224')
6 model = ViTForImageClassification.from_pretrained('google/vit-base-
7     patch16-224')

```

```

7 # Inference
8 from PIL import Image
9 image = Image.open('image.jpg')
10 inputs = processor(images=image, return_tensors="pt")
11 outputs = model(**inputs)

```

8. Bài tập tổng hợp

Ghi chú

Các bài tập sau giúp sinh viên *thực sự hiểu* sự khác biệt giữa các kiến trúc, không chỉ nhớ tên.

8.1 Bài tập lý thuyết

Bài tập

Bài 1. Giải thích trong 3 câu: tại sao skip connection của ResNet *vừa* giúp tránh vanishing gradient *vừa* giúp optimization dễ hơn?

Bài 2. So sánh *cơ chế kết nối* giữa ResNet (cộng) và DenseNet (nối). Mỗi cơ chế ảnh hưởng thế nào đến (a) số tham số, (b) memory, (c) khả năng overfit?

Bài 3. U-Net có skip connection *ngang* (giữa encoder và decoder), khác với skip connection *dọc* của ResNet. Vai trò của mỗi loại?

Bài 4. Tại sao YOLO không phải lúc nào cũng tốt hơn Faster R-CNN? Cho 2 ví dụ bài toán cụ thể: một bài toán mà YOLO tốt hơn, một bài toán mà Faster R-CNN tốt hơn.

Bài 5. ViT cần $\geq 10M$ ảnh để thắng CNN. Giải thích bằng khái niệm *inductive bias*.

Bài 6. Vì sao Swin Transformer có thể dùng làm backbone cho cả classification, detection, và segmentation, trong khi ViT thuần thì khó? Gợi ý: nghĩ về multi-scale feature.

8.2 Bài tập tình huống

Bài tập

Bài 7 (Chọn kiến trúc). Với mỗi tình huống sau, chọn kiến trúc phù hợp nhất và giải thích:

- (a) Bạn cần phát hiện và đếm số tế bào ung thư trong 500 ảnh hiển vi điện tử (mỗi tế bào cần một mask riêng).
- (b) Bạn xây hệ thống camera giám sát siêu thị, cần đếm số khách hàng vào ra real-time (30 FPS).

- (c) Bạn có dataset 50,000 ảnh X-quang ngực, mục tiêu là phát hiện và khoanh vùng vùng tổn thương phổi với độ chính xác cao nhất có thể.
- (d) Bạn làm pretraining vision model cho công ty internet lớn, có 100M ảnh từ web.
- (e) Bạn cần phân vùng đường xá trong ảnh vệ tinh độ phân giải 4096×4096 .

8.3 Bài tập thực hành

Bài tập

Bài 8 (Code). Dùng torchvision hoặc HuggingFace:

- (a) Load ResNet-50 và ViT-Base pretrained.
- (b) Phân loại cùng 5 ảnh test, so sánh top-5 predictions.
- (c) In ra số tham số của mỗi mô hình.
- (d) Đo thời gian inference trên CPU và GPU.

Bài 9 (Visualization). Dùng Grad-CAM (cho ResNet) và attention maps (cho ViT) trên cùng một ảnh. Hai mô hình “nhìn” cùng vùng không? Thảo luận.

9. Tổng kết

9.1 Sơ đồ tư duy cuối cùng

Khái niệm cốt lõi

Nếu chỉ nhớ **một câu duy nhất** về mỗi kiến trúc:

- **ResNet:** “Skip connection cứu deep learning.”
- **DenseNet:** “Nếu skip 1 layer tốt, skip mọi layer còn tốt hơn (cho data ít).”
- **U-Net:** “Encoder-decoder + skip ngang = segmentation tốt.”
- **Faster R-CNN:** “Hai giai đoạn = chính xác nhưng chậm.”
- **YOLO:** “Một lần forward = real-time.”
- **Mask R-CNN:** “Faster R-CNN + mask head = instance segmentation.”
- **ViT:** “Patch như word, attention thay convolution.”
- **Swin:** “Windowed attention = Transformer scale được lên ảnh lớn.”

9.2 Lời khuyên

Ghi chú

Một ít kinh nghiệm:

1. **Hiểu motivation, không học thuộc.** Mỗi kiến trúc ra đời để giải quyết một vấn đề cụ thể. Nếu bạn không hiểu vấn đề, bạn sẽ không biết khi nào nên dùng kiến trúc nào.
2. **Đừng chạy theo SOTA mù quáng.** Trên giấy, ViT-Huge thắng ResNet-50. Trong thực tế, khi bạn có 10,000 ảnh và 1 GPU, ResNet-50 sẽ tốt hơn nhiều.
3. **Skip connection xuất hiện ở mọi nơi.** Khi bạn thiết kế mạng mới, hãy nghĩ đến skip connection trước tiên. Nó là pattern thiết kế quan trọng nhất của thập kỷ vừa qua.

Hết bài giảng

Chúc các bạn nắm được bản đồ tư duy của Computer Vision hiện đại.

Tài liệu

- [He, Zhang, Ren, & Sun (2016)] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778). <https://arxiv.org/abs/1512.03385>
- [Huang, Liu, van der Maaten, & Weinberger (2017)] Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *CVPR* (pp. 4700–4708). <https://arxiv.org/abs/1608.06993>
- [Ronneberger, Fischer, & Brox (2015)] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *MICCAI* (pp. 234–241). <https://arxiv.org/abs/1505.04597>
- [Ren, He, Girshick, & Sun (2015)] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*. <https://arxiv.org/abs/1506.01497>
- [Redmon, Divvala, Girshick, & Farhadi (2016)] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, real-time object detection. In *CVPR* (pp. 779–788). <https://arxiv.org/abs/1506.02640>
- [He, Gkioxari, Dollár, & Girshick (2017)] He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask R-CNN. In *ICCV* (pp. 2961–2969). <https://arxiv.org/abs/1703.06870>

- [Dosovitskiy et al. (2021)] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2010.11929>
- [Liu et al. (2021)] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). Swin Transformer: Hierarchical vision transformer using shifted windows. In *ICCV* (pp. 10012–10022). <https://arxiv.org/abs/2103.14030>